Datadog Deployment for Hybrid & Cloud Estates

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Datadog Deployment for Hybrid & Cloud Estates 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Business value . . . 5

Effort and boundary . . . 5

1. Solution Overview — Datadog Deployment for Hybrid & Cloud Estates 5

What this solution is . . . 5

How the technology works . . . 5

What this solution builds (the components) . . . 6

The standard deployment model . . . 6

Scope . . . 7

What is out of scope . . . 7

Authentication and connections used . . . 7

Limitations (be honest with buyers) . . . 7

2. Requirements Guide — Datadog Deployment for Hybrid & Cloud Estates 8

2.1 A Datadog organization on the correct site, with API + APP keys . . . 8

2.2 Datadog Terraform provider + version pinned . . . 8

2.3 Agent rollout channel for hybrid + cloud hosts . . . 8

2.4 Secret store for API/APP keys and integration credentials . . . 9

2.5 Azure integration prerequisites (if monitoring Azure) . . . 9

2.6 AWS integration prerequisites (if monitoring AWS) . . . 9

2.7 A defined pilot host + cluster and notification targets . . . 9

2.8 Terraform + shell/PowerShell tooling on the operator workstation . . . 9

Checklist . . . 10

Global Requirements — Observability (category-level) 10

G1. Target cloud/estate with sufficient rights . . . 10

G2. A telemetry destination (workspace / monitoring account) . . . 10

G3. Agent / collection rollout channel . . . 10

G4. IaC tooling and source control . . . 11

G5. Secrets management . . . 11

G6. Ingestion cost plan . . . 11

G7. Alerting and notification targets . . . 11

G8. A pilot scope . . . 11

3. Implementation Guide — Datadog Deployment for Hybrid & Cloud Estates 11

Step 1 — Assess: inventory the estate and confirm access . . . 12
Step 2 — Configure: pin the provider and set backend + variables (no keys in code) . . . 12
Step 3 — Pilot: install the agent on the pilot hosts . . . 13
Step 4 — Pilot: wire the cloud integrations . . . 13
Step 5 — Pilot: apply dashboard, SLO, monitors, and log pipeline (plan first) . . . 13
Step 6 — Validate the pilot (the gates) . . . 14
Step 7 — Expand: roll out across the estate in waves . . . 14
5. Testing & Validation — Datadog Deployment for Hybrid & Cloud Estates 14
Test matrix . . . 14
Automated checks . . . 15
Evidence to capture . . . 16
Exit criteria (before Expand / estate-wide rollout) . . . 16
6. Troubleshooting — Datadog Deployment for Hybrid & Cloud Estates 16
Agent installs but the host doesn't appear in Datadog . . . 16
Agent reports but has no/incorrect tags . . . 16
Terraform provider auth fails . . . 17
Azure integration not connecting . . . 17
AWS integration not connecting . . . 17
A key was found in code or state . . . 17
Monitor never fires . . . 18
Monitor fires but no notification . . . 18
Logs arrive unparsed or in the wrong index . . . 18
Ingestion / host count higher than expected (cost) . . . 18
7. Best Practices — Datadog Deployment for Hybrid & Cloud Estates 19
Manage everything as code . . . 19
Never put keys in code . . . 19
Tag first — tags are the whole game . . . 19
Monitors that page, not spam . . . 19
Control cost deliberately (Datadog-side decisions) . . . 20
Roll out safely . . . 20
Where to go next (adjacent catalog areas) . . . 20
8. Deliverables Summary — Datadog Deployment for Hybrid & Cloud Estates 20
What you receive . . . 20
Outcomes . . . 22
Assumptions & boundaries . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Datadog Deployment for Hybrid & Cloud Estates

The problem

Teams running **Azure, AWS, and on-prem** at the same time end up with **fragmented monitoring**: the Azure portal shows Azure, CloudWatch shows AWS, some VMs have an agent and some don't, Kubernetes is a black box, and there is no single place to answer "is the service healthy end-to-end?" Monitors get hand-clicked in one console, never versioned, and drift. When an incident spans clouds, responders stitch together three tools and a Slack thread. There is **no one pane of glass**, and nothing is **repeatable**.

The solution

This solution deploys **Datadog across the whole estate — managed as code** — so observability is unified, versioned, and repeatable:

1. **Agent rollout as code** — the Datadog Agent deployed consistently to **VMs (Linux + Windows), containers, and Kubernetes, hybrid and cloud, with the API key sourced from a** secret store** (not baked into images or scripts).
1. **Cloud integrations** — **Azure**, **AWS**, and **Kubernetes** integrations wired up so Datadog pulls metrics, logs, and inventory from each cloud's native services, not just the hosts.
1. **Curated dashboards + SLOs** — an estate-overview dashboard and **Service Level Objectives** defined as code, so the important views and targets exist by default and survive turnover.
1. **Monitors + alerting** — monitors with **tuned thresholds** and **notification routing** (Slack/Teams/PagerDuty/email), started on a pilot and widened deliberately — not a wall of noisy defaults.
1. **Log pipelines + APM foundations** — a log-ingestion pipeline (parsing, tags, indexes) and the groundwork for Application Performance Monitoring, all in **Terraform**.

Delivery follows a disciplined **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model: inventory the estate and confirm access, author the Terraform and agent config, roll agents + monitors to a **pilot host/cluster** first, validate that data and alerts flow, then expand across the estate in waves — every change a reviewed `terraform plan` before `apply`.

What's included

Full documentation (overview, requirements, ordered implementation, testing matrix, troubleshooting, best practices) plus **five deployable artifacts**: the core **Terraform** (`main.tf`, Datadog provider — integrations, dashboard, monitors, SLOs, log pipeline), a **Linux agent installer** (`install-datadog-agent.sh`), a **Windows agent installer** (`Install-DatadogAgent.ps1`), a **validation** script (`OBSDDOG-Validate.ps1`), and an **inventory/coverage report** (`OBSDDOG-Report.ps1`). Everything is **plan-first and pilot-first**; **API and APP keys come from a**

secret store and never appear in code.

Business value

- **One pane of glass** — Azure + AWS + on-prem in a single view, correlated by tags.
- **Repeatable & versioned** — monitors, dashboards, and SLOs are Terraform, reviewed as pull requests.
- **Tuned, not noisy** — thresholds proven on a pilot before they page the whole team.
- **Secretless** — keys live in a vault/secret store; the repo is safe to share.

Effort and boundary

A competent admin comfortable with Terraform and package management can stand up the pilot (agent on a few hosts + a cluster, the cloud integrations, one dashboard, a couple of monitors, one SLO) in a few focused sessions, then expand. This guide provides the **technical deployment and the automation**. Your organization's **monitor thresholds, SLO targets, on-call routing, log retention/indexing, and which Datadog products to license** are operational/commercial decisions the guide wires up but does not make.

> ***Product & pricing note:*** *Datadog's product SKUs, per-host/ingestion **pricing**, agent versions, Terraform provider resources, and console UI **change over time**, and the internals of Datadog billing/plans are **out of scope** — deferred to Datadog's own documentation. Every version- or pricing-sensitive point here is flagged **"verify"** — confirm current provider versions, agent releases, and plan/SKU availability against **Datadog's docs** before broad rollout.*

1. Solution Overview — Datadog Deployment for Hybrid & Cloud Estates

What this solution is

Datadog is a SaaS observability platform that unifies metrics, logs, traces (APM), and events from across an estate into one place. This solution **deploys Datadog across a hybrid + multicloud estate as code** — the agent on every host/container/cluster, the cloud integrations for Azure/AWS/Kubernetes, and the dashboards, SLOs, monitors, and log pipelines defined in **Terraform** so the whole observability stack is versioned and repeatable. The Datadog console's internals and its pricing/plans are **deferred to Datadog's own documentation**; this solution is the **deployment and management-as-code** layer.

How the technology works

- **The Datadog Agent** runs on each host (VM or container node) and collects system metrics, logs, and

(with APM) traces, forwarding them to your Datadog **organization** (a regional Datadog site). It authenticates with a **Datadog API key**. It is configured via `datadog.yaml` and per-integration config files (or, on Kubernetes, a Helm chart / DaemonSet).

- **Cloud integrations** pull telemetry Datadog can't see from a host: the **Azure** integration uses an

Entra **app registration** (service principal) with **Monitoring Reader**; the **AWS** integration uses a cross-account **IAM role**; the **Kubernetes** integration uses the cluster agent. These pull cloud-service metrics, inventory, and (optionally) logs.

• **The Datadog Terraform provider** manages Datadog *resources* — dashboards, monitors, SLOs, log

pipelines, integrations — via the Datadog API, authenticated with an **API key + APP key**. `terraform plan` shows what will change; `terraform apply` makes it so. Because it is code in Git, monitors and dashboards are reviewed, versioned, and reproducible.

• **Tags** are the connective tissue — `env`, `service`, `team`, cloud/region — applied at the agent and

integration level so one dashboard/monitor/SLO can span Azure + AWS + on-prem.

• **Monitors** evaluate a metric/log/APM condition and notify via **@-handles** (Slack, Teams, PagerDuty,

email). **SLOs** track a target (e.g. 99.9% availability) over a window. **Log pipelines** parse and enrich incoming logs and route them to **indexes** (which control retention/cost).

What this solution builds (the components)

1. **Agent rollout as code**

• Linux (`install-datadog-agent.sh`) and Windows (`Install-DatadogAgent.ps1`) installers that read

the **API key from a secret store**, set the **site** and base **tags**, and enable logs/APM as chosen; Kubernetes via the official **Helm chart / DaemonSet** (referenced from Terraform/CI).

1. **Cloud integrations**

• **Azure** (`datadog_integration_azure`), **AWS** (`datadog_integration_aws`), and **Kubernetes**

wired in Terraform, with the credentials/role **referenced from a secret store**, never inline.

1. **Curated dashboards + SLOs**

• An estate-overview dashboard `OBSDDOG-Dashboard-EstateOverview` and an SLO

`OBSDDOG-SLO-APIAvailability`, defined as code.

1. **Monitors + alerting**

• Monitors such as `OBSDDOG-Monitor-HostUnreachable-Pilot`, `OBSDDOG-Monitor-CPUHigh-Pilot`, and

`OBSDDOG-Monitor-K8sPodCrashloop-Pilot`, with tuned thresholds and **notification routing**, started on the **pilot** scope.

1. **Log pipeline + APM foundation**

• A log pipeline `OBSDDOG-Pipeline-AppLogs` (parsing + tags + index) and the APM enablement flags,

ready to extend to instrumented services.

The standard deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand*** *— inventory hosts/clusters/clouds and confirm access and keys (Assess); author the Terraform + agent config and pin provider/agent versions (Configure); install the agent and apply integrations/monitors against a* ***pilot host + cluster*** *first,* `plan` *before* `apply` *(Pilot); prove metrics/logs flow and a monitor fires and routes (Validate); then roll agents and monitors across the estate in* ***waves****, widening thresholds deliberately (Expand). Every change is a reviewed* `terraform plan`*.*

Scope

• Deploy the **Datadog Agent** across VMs, containers, and Kubernetes (hybrid + cloud) as code.

• Wire the **Azure / AWS / Kubernetes** cloud integrations.

• Deliver a **dashboard**, at least one **SLO**, a curated set of **monitors** with routing, and a **log pipeline + APM foundation** — all in Terraform.

What is out of scope

• **Datadog console internals, plan/SKU selection, and pricing** — deferred to Datadog's documentation;

this solution deploys and manages, it does not choose your Datadog commercial plan.

• The **decisions** of monitor thresholds, SLO targets, on-call routing, and log retention/indexing — the guide encodes your choices; the choices are the service owners'.

• **Deep application instrumentation** (adding APM tracing libraries into every service's source) — the APM *foundation* is enabled; instrumenting each app is a per-service development task.

• **Azure-native-only** observability — if you are Azure-only, the **Azure Monitor** solution in this category is the lighter-weight, first-party option.

Authentication and connections used

• **Datadog API key** — used by the **agent** to submit data; sourced from a **secret store** at install time, never written into images/scripts/repo.

• **Datadog API key + APP key** — used by the **Terraform provider** to manage Datadog resources; supplied via environment variables / secret store, never in `.tf` or state-in-repo.

• **Azure integration** — an Entra **app registration** with **Monitoring Reader**; **AWS integration** — a cross-account **IAM role**; both referenced from a secret store.

Limitations (be honest with buyers)

• **Datadog bills on hosts, ingestion, and per-product** — an uncontrolled rollout (all logs, all hosts, every product) is expensive. Collect and index deliberately; that plan is a Datadog-side decision (*verify current pricing/SKUs in Datadog's docs*).

• **Version sensitivity** — the agent, the Terraform provider, and integration behavior change; all versions here are **pinned and flagged "verify."**

• `terraform apply` **changes real monitoring config.** Everything is **plan-first** and **pilot-first**; keys are **never** in code — the repo must be safe to share.

2. Requirements Guide — Datadog Deployment for Hybrid & Cloud Estates

Verify **every** requirement before you deploy. Each entry: **Description · Why required · How to verify · How to implement if missing.** The category-wide prerequisites (a target cloud/estate with rights, a telemetry destination i.e. a Datadog org, an at-scale rollout channel, IaC tooling + source control, secrets management, an ingestion-cost plan, notification targets, and a pilot scope) are defined once in `global-requirements-observability.md` — confirm those first. The items below are **specific to deploying Datadog across a hybrid/multicloud estate**.

2.1 A Datadog organization on the correct site, with API + APP keys

Description: A Datadog **organization** provisioned on the right **site** (e.g. US1/US3/US5/EU/AP — *verify your org's site*), with a **API key** (agent + provider) and an **Application (APP) key** (provider), issued to a service/automation identity. **Why required:** The agent submits data with the API key; the Terraform provider manages resources with API + APP keys. The site determines the endpoints the agent and provider talk to. **How to verify:** You can sign in to the Datadog org; the API/APP keys exist under **Organization Settings** → **API Keys / Application Keys**; `OBSDDOG-Validate.ps1` confirms provider auth. **How to implement if missing:** Create the org (Datadog-side), generate an API key and an APP key for automation, and record the **site**. Store both keys in the **secret store** (§2.4). *Datadog plan/SKU selection is deferred to Datadog's docs.*

2.2 Datadog Terraform provider + version pinned

Description: The `DataDog/datadog` Terraform provider, **pinned** to a supported version, plus Terraform CLI, recorded in the repo. **Why required:** The dashboards/monitors/SLOs/integrations are provider resources; an unpinned provider upgrade can change plan behavior mid-rollout. **How to verify:** `terraform version` and the `required_providers` block match; `terraform init` resolves the provider. *Verify the current supported provider version against the registry/Datadog docs.* **How to implement if missing:** Pin `datadog` in `main.tf` (example provided, flagged "verify"), commit `.terraform.lock.hcl`.

2.3 Agent rollout channel for hybrid + cloud hosts

Description: A way to deploy the agent at scale: a **config-management/IaC channel** (Ansible/Chef, Intune/Configuration Manager, cloud-init/user-data) for VMs, and **Helm/DaemonSet** for Kubernetes — plus network egress from hosts to the Datadog **site** endpoints. **Why required:** Hand-installing the agent per host doesn't scale or stay consistent; the installers here are the per-host action the rollout channel invokes. **How to verify:** A pilot host can run `install-datadog-agent.sh` / `Install-DatadogAgent.ps1` and reach Datadog (agent status = OK); the K8s cluster can pull the

Datadog Helm chart. **How to implement if missing:** Establish the rollout channel and confirm outbound network/proxy access to the Datadog site before onboarding hosts.

2.4 Secret store for API/APP keys and integration credentials

Description: A secure store — **Key Vault / AWS Secrets Manager / pipeline secrets** — holding the Datadog **API key**, **APP key**, the Azure integration **app-registration secret**, and the AWS **role** details. **Never in code, images, or state committed to the repo. Why required:** These keys grant broad read (and monitoring write); a committed key leaks the estate's observability and can incur cost/abuse. **How to verify:** No keys in the repo (secret scanning passes); the installers and Terraform read keys from env/secret references at runtime. **How to implement if missing:** Put every key in the secret store; wire the installers and CI to read them; enable secret scanning; store Terraform **state** in a secured backend (not the repo).

2.5 Azure integration prerequisites (if monitoring Azure)

Description: An Entra **app registration** (service principal) with the **Monitoring Reader** (and **Reader**) role at the target subscription(s)/management group, and its client secret in the secret store. **Why required:** The Datadog Azure integration authenticates as this app to pull Azure metrics and inventory. **How to verify:** The app has Monitoring Reader at scope; `OBSDDOG-Validate.ps1` (or the Datadog Azure integration status page) shows the subscription connected. **How to implement if missing:** Create the app registration, assign Monitoring Reader at scope, generate a client secret (to the secret store), and register it in the `datadog_integration_azure` resource.

2.6 AWS integration prerequisites (if monitoring AWS)

Description: A cross-account **IAM role** Datadog can assume (Datadog's account as trusted principal with the standard integration policy), in each target AWS account. **Why required:** The Datadog AWS integration assumes this role to read CloudWatch metrics, tags, and inventory. **How to verify:** The role exists with the correct trust + policy; the Datadog AWS integration status shows the account connected. **How to implement if missing:** Create the IAM role/policy per Datadog's AWS integration guidance and reference it in the `datadog_integration_aws` resource. *Verify the current required policy in Datadog's docs.*

2.7 A defined pilot host + cluster and notification targets

Description: A **pilot** set — at least one Linux VM, one Windows VM (if applicable), and one Kubernetes cluster/namespace — plus the concrete **notification handles** (Slack/Teams channel, PagerDuty service, email) for the pilot monitors. **Why required:** Agents, integrations, monitor thresholds, and routing are validated on the pilot before estate-wide rollout. **How to verify:** The pilot hosts/cluster exist and are reachable; the Slack/PagerDuty integration is connected in Datadog and a test notification succeeds. **How to implement if missing:** Nominate the pilot scope and connect the notification integrations in Datadog. *Thresholds, SLO targets, and on-call routing are the service owner's decisions; the guide wires them up.*

2.8 Terraform + shell/PowerShell tooling on the operator workstation

Description: Terraform CLI, **Git**, a **bash** shell (for the Linux installer / plan runs), and **PowerShell** (for the Windows installer and the validate/report helpers). **Why required:** The stack deploys from Terraform; the

helpers and installers are bash/PowerShell. **How to verify:** `terraform version`, `git --version`, `bash --version`, and `pwsh --version` all work. **How to implement if missing:** Install Terraform, Git, and PowerShell; ensure a bash environment is available for the Linux installer.

Checklist

- Category global requirements confirmed (`global-requirements-observability.md`)
- Datadog org on the correct **site**; API + APP keys issued to automation — *verify site*
- `DataDog/datadog` Terraform provider **pinned** — *verify current version*
- Agent rollout channel ready (config-mgmt/IaC for VMs, Helm/DaemonSet for K8s) + network egress
- Secret store holds API/APP keys + integration creds; **no keys in code**; state secured
- Azure integration: app registration + **Monitoring Reader** (if monitoring Azure)
- AWS integration: cross-account **IAM role** (if monitoring AWS)
- Pilot host + cluster nominated; Slack/Teams/PagerDuty notification targets connected
- Terraform + Git + bash + PowerShell installed

Global Requirements — Observability (category-level)

These requirements apply to every solution in the Observability category (Azure Monitor / Log Analytics, Datadog, and related telemetry/monitoring platforms). Published once at the category level so listings stay small. Confirm before deploying any observability solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Observability collects telemetry across the estate — collect what you will act on, control ingestion cost deliberately, and roll agents/diagnostics out in a ring before fleet-wide. This is ops/platform monitoring, distinct from security SIEM (see the Sentinel category).*

G1. Target cloud/estate with sufficient rights

Description: The Azure subscription(s), AWS account(s), and/or hosts to be monitored, with rights to configure diagnostics and deploy agents. Why: Monitoring configuration (diagnostic settings, agents, integrations) needs contributor-level access at the target scope. Verify: `az account show` (Azure) / access to the target hosts succeeds with the needed role. Implement: Obtain Contributor (or scoped monitoring) rights at the target scope.

G2. A telemetry destination (workspace / monitoring account)

Description: A Log Analytics workspace (Azure Monitor) or a Datadog organization/account. Why: Telemetry needs a governed destination with retention and access control. Verify: The workspace/Datadog org exists and is access-controlled. Implement: Provision the workspace / Datadog account before wiring sources to it.

G3. Agent / collection rollout channel

Description: A way to deploy agents and collection config at scale — Azure Policy + Azure Monitor Agent / Data Collection Rules, or a config-management/IaC channel for the Datadog agent. Why: Manual per-host agent installs don't scale and drift. Verify: The rollout channel (Policy/DCR, Intune/CM, Ansible, Terraform) is available. Implement: Establish the at-scale rollout mechanism before onboarding hosts.

G4. IaC tooling and source control

Description: Terraform / Bicep, the relevant CLI (Az / Datadog), and Git. Why: The observability stack is delivered as versioned code so it is repeatable and reviewable. Verify: `terraform version` / `az version` and `git --version` return supported builds. *Verify current provider versions.* Implement: Install and pin the toolchain; store config in Git.

G5. Secrets management

Description: A secure store for API/app keys (Datadog API+APP keys, service principals) — Key Vault / pipeline secrets, never in code. Why: Monitoring keys grant broad read (and some write); committed keys leak. Verify: No keys in the repo; the pipeline reads them from a secret store. Implement: Move keys to Key Vault / secret store; enable secret scanning.

G6. Ingestion cost plan

Description: An understanding of expected data volume and a cost/retention plan (table tiers, sampling, filters). Why: Observability platforms bill on ingestion/retention; uncontrolled collection is expensive. Verify: A volume estimate and retention decision exist. *Verify current pricing/tier availability.* Implement: Define what to collect vs drop, retention, and (Azure) table tiers before broad rollout.

G7. Alerting and notification targets

Description: Notification channels (email, Teams/Slack, PagerDuty) and an on-call/routing model. Why: Alerts are only useful if they reach the right responder with context. Verify: Action groups / Datadog notification integrations and routing are defined. Implement: Configure notification channels and routing before enabling alert rules.

G8. A pilot scope

Description: A defined pilot set of resources/hosts to onboard first. Why: Diagnostics/agents and alert thresholds are validated on a pilot before fleet-wide rollout. Verify: A pilot resource group / host set exists. Implement: Define the pilot scope before scaling collection and alerting. *Alert thresholds and SLOs are the service owner's decision; this guide provides the technical wiring.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Datadog Deployment for Hybrid & Cloud Estates

Follow these steps **in order**. Work in your **Git repo** (Terraform + agent config), a **terminal** (agent installers + `terraform plan/apply`), the **Datadog console** (verification + connecting notification integrations), and your **cloud portals** (integration prerequisites). Names in `code` are the exact objects to create — keep them consistent so the Terraform, monitors, dashboards, and SLOs all refer to the same objects. Provider/agent versions and any Datadog SKU/pricing points are flagged **"verify."**

Object naming convention: `<Purpose>-<Scope>-<Stage>`. The objects you build:

Object	Name
Estate-overview dashboard	`OBSDDOG-Dashboard-EstateOverview`
SLO (API availability)	`OBSDDOG-SLO-APIAvailability`
Monitor — host unreachable	`OBSDDOG-Monitor-HostUnreachable-Pilot`
Monitor — CPU high	`OBSDDOG-Monitor-CPUHigh-Pilot`
Monitor — K8s pod crashloop	`OBSDDOG-Monitor-K8sPodCrashloop-Pilot`
Log pipeline	`OBSDDOG-Pipeline-AppLogs`
Azure integration	`datadog_integration_azure` (per subscription)
AWS integration	`datadog_integration_aws` (per account)
Base agent tags	`env:<env>`, `service:<svc>`, `team:<team>`

> ***Model:*** *Assess → Configure →* ***Pilot on one host + one cluster first*** *→ Validate → Expand (estate-wide in waves). Every change is a reviewed* `terraform plan` ***before*** `apply`*.* ***API/APP keys come from the secret store — never in code or state committed to the repo.*** *Rollback guidance is at the end.*

Step 1 — Assess: inventory the estate and confirm access

1. Inventory hosts, clusters, and cloud accounts to be monitored; identify the **pilot** (one Linux VM, one Windows VM if applicable, one Kubernetes cluster/namespace).

1. Confirm the **Datadog org + site** and that the **API key** and **APP key** are in the **secret store** (§2.1, §2.4). Confirm the Azure app registration / AWS role prerequisites (§2.5–2.6) exist if monitoring those clouds.

1. Confirm outbound **network egress** from pilot hosts to the Datadog site endpoints.

Step 2 — Configure: pin the provider and set backend + variables (no keys in code)

1. In your repo, add the provided `main.tf` (Datadog provider + integrations, dashboard, monitors, SLO, log pipeline). **Pin** the `DataDog/datadog` provider version and run `terraform init`; commit `.terraform.lock.hcl`. *Verify the current supported provider version.*

1. Configure the Datadog provider to read `DD_API_KEY`, `DD_APP_KEY`, and the **site** from

environment variables / secret store — do **not** put keys in `.tf`. Store Terraform **state** in a secured backend (not the repo).

1. Set your inputs (env/service/team tags, pilot host filter, notification handles, thresholds, SLO target) in `variables.tf` / `terraform.tfvars`.

Step 3 — Pilot: install the agent on the pilot hosts

1. **Linux:** on the pilot Linux VM, run `install-datadog-agent.sh` — it reads the **API key from the secret store **(env var / vault call), sets the** site **and base** tags **(**`env`**,** `service`**,** `team`**), optionally enables** logs **and** APM**, installs the agent, and starts it.

```
DD_API_KEY="$(read-from-secret-store)" DD_SITE="datadoghq.com" \
  ./install-datadog-agent.sh --env pilot --service demo --team platform --logs --apm
```

1. **Windows:** on the pilot Windows VM, run `Install-DatadogAgent.ps1` with the same inputs (API key from the secret store, site, tags).

1. **Kubernetes:** install the Datadog **Helm chart** (or apply the DaemonSet) on the pilot cluster with the API key from a **Kubernetes secret** (never a literal) and the same base tags.

1. **Confirm the agent reports:** in Datadog → **Infrastructure** → **Host Map / Containers**, the pilot host(s) and cluster appear with recent data; `datadog-agent status` on a host shows **OK**.

Step 4 — Pilot: wire the cloud integrations

1. **Azure:** in `main.tf`, configure `datadog_integration_azure` with the app-registration `client_id`/tenant and the **secret from the secret store**; `terraform plan` → review → `apply`.

1. **AWS:** configure `datadog_integration_aws` with the cross-account **IAM role**; `plan` → `apply`.

2. **Kubernetes:** confirm the cluster agent integration is reporting (it comes with the Helm install).

3. **Confirm in Datadog:** the Azure/AWS integration status pages show the subscription/account **connected** and metrics arriving.

Step 5 — Pilot: apply dashboard, SLO, monitors, and log pipeline (plan first)

1. Run `terraform plan -out tfplan` and **read the what-if**: it should create `OBSDDOG-Dashboard-EstateOverview`, `OBSDDOG-SLO-APIAvailability`, the three **pilot** monitors (`OBSDDOG-Monitor-HostUnreachable-Pilot`, `-CPUHigh-Pilot`, `-K8sPodCrashloop-Pilot`), and `OBSDDOG-Pipeline-AppLogs` — and **nothing unexpected**.

1. `terraform apply tfplan`.

2. Confirm the monitors are **scoped to the pilot** (via the `env:pilot` / host tag filter) and their **notification handles** (`@slack-…` / `@pagerduty-…`) are set — send a monitor **test notification**.

1. Confirm the **log pipeline** parses pilot logs and routes them to the intended **index**.

Step 6 — Validate the pilot (the gates)

Run `OBSDDOG-Validate.ps1` and `OBSDDOG-Report.ps1` and confirm the four gates in §5 pass: agents report, integrations connected, a monitor fires and routes, and the dashboard/SLO render. **Pass** = all gates green → proceed to Expand. **Fail** = fix before scaling (see §6).

Step 7 — Expand: roll out across the estate in waves

1. **Agents:** drive the installers through the **rollout channel** (config-mgmt/IaC for VMs,

Helm/DaemonSet for the remaining clusters) in **waves** — a host group / environment at a time — keeping the API key sourced from the secret store.

1. **Monitors + SLOs:** widen monitor scopes from `env:pilot` to broader tag scopes (or clone per

critical service), **tuning thresholds** as real data arrives; add SLOs for the services that need them. Every change is a reviewed `plan → apply`.

1. **Integrations:** add the remaining Azure subscriptions / AWS accounts / clusters to the integration

resources.

1. Re-run `OBSDDOG-Report.ps1` after each wave to capture host/monitor/SLO coverage and watch

ingestion against the cost plan.

> ***Rollback / safety (any step):*** *Terraform is your rollback for Datadog resources —* ***revert the commit*** *and* `apply` *the previous known-good state (a bad monitor/dashboard is one revert away). To stop paging without deleting,* ***mute*** *or set a monitor to* `no-data`*/downtime, or narrow its scope. To remove the agent from a host, run the installer's* ***uninstall*** *path (or the OS package removal). To disconnect a cloud, remove the integration resource and revoke the app-registration/role.* ***Roll out in waves under change control*** *— never install estate-wide in one shot (cost + notification-storm risk).* ***Never*** *hard-code a key to "make it work" — fix the secret-store reference instead.*

Order of operations

```
Assess (inventory + keys in secret store) → pin provider + secure backend/vars
(no keys in code) → pilot: install agent (Linux/Windows/K8s) → wire
Azure/AWS/K8s integrations → plan/apply dashboard + SLO + pilot monitors + log
pipeline → validate the 4 gates → expand agents + monitors + integrations
estate-wide in waves
```

5. Testing & Validation — Datadog Deployment for Hybrid & Cloud Estates

Validate against the **pilot** (one host + one cluster + the connected clouds) first. Each test has a **pass/fail** signal. Do not expand agents or widen monitors estate-wide until all four gates pass. Reference scripts are named in each row. Everything is **plan-first**; **no keys in code**.

Test matrix

#	Gate	What you do	Pass signal	Fail signal
1	**G1 Agent reports**	`datadog-agent status` on the pilot host; check Infrastructure → Host Map	Pilot Linux/Windows host + cluster show **recent** metrics; status = OK	Host absent / agent error / no data
2	**G1 Tags applied**	Inspect a pilot host's tags in Datadog	`env`, `service`, `team` tags present as set by the installer	Missing/incorrect tags (dashboards/monitors won't scope)
3	**G2 Azure integration**	`OBSDDOG-Validate.ps1`; Datadog Azure integration status	Subscription **connected**; Azure metrics arriving	Not connected / app-registration/role error
4	**G2 AWS integration**	Datadog AWS integration status	Account **connected**; CloudWatch metrics arriving	Role assume fails / not connected
5	**G2 No keys in code**	Secret scan the repo; inspect `main.tf`/installers	Keys read from env/secret store only; scan clean	An API/APP key or secret found in code/state
6	**G3 Monitor fires**	Trigger a pilot condition (stop a host / spike CPU)	`OBSDDOG-Monitor-*-Pilot` fires; state changes to Alert	Monitor never triggers
7	**G3 Notification routes**	Confirm the fired monitor's notification	Arrives in the mapped **Slack/Teams/PagerDuty** handle	No notification / wrong channel
8	**G3 Log pipeline**	Send a pilot app log	`OBSDDOG-Pipeline-AppLogs` parses it; it lands in the intended index with tags	Unparsed / wrong index
9	**G4 Dashboard + SLO**	Open `OBSDDOG-Dashboard-EstateOverview` and `OBSDDOG-SLO-APIAvailability`	Dashboard renders with pilot data; SLO shows a status/error budget	Blank / query errors / SLO not tracking

Automated checks

• `OBSDDOG-Validate.ps1` — confirms Terraform provider auth works (API+APP keys present via env, not

in code), the expected Datadog resources exist (dashboard, SLO, the three pilot monitors, log pipeline), the Azure/AWS integrations report connected, and **no keys are present in the repo/state**. **Read-only.**

• `OBSDDOG-Report.ps1` — inventory/coverage report via the Datadog API: hosts reporting (by

env/service tag), monitors and their states, SLO status/error budget, and an ingestion snapshot to watch against the cost plan. **Read-only.**

• `install-datadog-agent.sh` **/** `Install-DatadogAgent.ps1` — the agent installers themselves; their

`status` output is the Gate-1 evidence per host.

• `main.tf` — `terraform plan` is the primary what-if gate for every Datadog-resource change.

Evidence to capture

• `datadog-agent status` (Linux + Windows) and a Host Map screenshot showing the pilot reporting.

• Datadog **Azure** and **AWS** integration status pages showing *connected*.

• A **secret-scan** report showing the repo/state is key-free.

• A fired-monitor screenshot **and** the received Slack/Teams/PagerDuty notification.

• `OBSDDOG-Report.ps1` output (hosts, monitors, SLO) — the coverage baseline before Expand.

Exit criteria (before Expand / estate-wide rollout)

• G1 — pilot agents report with correct `env`/`service`/`team` tags

• G2 — Azure + AWS + K8s integrations connected; **secret scan clean (no keys in code/state)**

• G3 — a monitor fires and routes to the correct notification handle; log pipeline parses to index

• G4 — estate-overview dashboard renders and the SLO tracks an error budget

• `OBSDDOG-Validate.ps1` and `OBSDDOG-Report.ps1` run clean; ingestion within the cost plan

6. Troubleshooting — Datadog Deployment for Hybrid & Cloud Estates

Symptom → root cause → fix for the failures you are most likely to hit. Re-run `OBSDDOG-Validate.ps1` and `OBSDDOG-Report.ps1` after each fix to confirm. Work on the **pilot** first. Never resolve an issue by hard-coding a key — fix the **secret-store reference** instead.

Agent installs but the host doesn't appear in Datadog

• **Symptom:** `install-datadog-agent.sh` / `Install-DatadogAgent.ps1` completes, but the host isn't on

the Host Map.

• **Cause:** Wrong/empty **API key**, wrong **site** (agent talking to the wrong region), or blocked

outbound network to the Datadog endpoints.

• **Fix:** Run `datadog-agent status` — it shows the API-key and endpoint status. Confirm the key was

read from the **secret store** and the `DD_SITE` matches your org's site. Verify egress/proxy to the Datadog site endpoints (`datadog-agent status` reports connectivity). *Verify current endpoints for your site in Datadog's docs.*

Agent reports but has no/incorrect tags

- **Symptom:** Host is present but dashboards/monitors scoped by `env:pilot` don't see it.
- **Cause:** The installer's `env`/`service`/`team` tags weren't passed, or `datadog.yaml`/host tags differ from what the monitors filter on.
- **Fix:** Re-run the installer with the correct `--env`/`--service`/`--team` (or fix `tags:` in `datadog.yaml`) and restart the agent; confirm the tags on the host in Datadog match the monitor filters exactly.

Terraform provider auth fails

- **Symptom:** `terraform plan` errors with 403/authentication or "API key invalid."
- **Cause:** Missing/invalid `DD_API_KEY` or `DD_APP_KEY` env vars, wrong **site**, or the APP key belongs to a different org/user.
- **Fix:** Export both keys from the **secret store** into the environment (not into `.tf`), set the provider `api_url`**/site** correctly, and confirm the APP key is valid for the target org. Never commit keys to fix this.

Azure integration not connecting

- **Symptom:** The Datadog Azure integration status shows the subscription not connected / no metrics.
- **Cause:** The app registration lacks **Monitoring Reader** at scope, the client secret is wrong/expired, or the wrong tenant/subscription ID.
- **Fix:** Confirm the app registration has **Monitoring Reader** (and Reader) at the target scope, regenerate the client secret into the **secret store**, and re-`apply` the `datadog_integration_azure` resource with the correct tenant/subscription IDs. *Verify current required roles in Datadog's docs.*

AWS integration not connecting

- **Symptom:** AWS integration status shows the account not connected.
- **Cause:** The cross-account **IAM role** trust/policy is wrong, or the external ID / role ARN doesn't match what the `datadog_integration_aws` resource declares.
- **Fix:** Verify the IAM role trusts Datadog's account with the correct external ID and carries the current integration policy; align the role ARN in Terraform and re-`apply`. *Verify the current policy in Datadog's docs.*

A key was found in code or state

- **Symptom:** Secret scan (Gate 5) flags an API/APP key or client secret in `main.tf`, an installer, or committed state.
- **Cause:** A key was inlined instead of referenced from the secret store, or state was committed to the

repo.

- **Fix:** Remove the key, **rotate it immediately** (it is compromised once committed), move it to the secret store, purge it from Git history, and move Terraform **state** to a secured backend. Re-run the secret scan.

Monitor never fires

- **Symptom:** A pilot condition is clearly true but the monitor stays green.
- **Cause:** The monitor's **query/scope** doesn't match the pilot tags, the threshold/window is too lax, or it's evaluating a metric the agent isn't sending.
- **Fix:** Open the monitor, run its **query** in the metric explorer, and confirm it returns pilot data; align the **scope tags** and **threshold/window**; ensure the underlying metric is being collected (agent integration enabled).

Monitor fires but no notification

- **Symptom:** Monitor goes to Alert but no Slack/Teams/PagerDuty message.
- **Cause:** The notification **@-handle** is wrong/not connected, or the integration (Slack/PagerDuty) isn't authorized in Datadog.
- **Fix:** Confirm the Slack/Teams/PagerDuty integration is **connected** in Datadog and the monitor message uses the correct `@handle`; send a **test notification** from the monitor.

Logs arrive unparsed or in the wrong index

- **Symptom:** Pilot logs show up as raw text or in an unexpected index.
- **Cause:** `OBSDDOG-Pipeline-AppLogs` parsing rules don't match the log format, or the index filter/retention routes them elsewhere.
- **Fix:** Adjust the pipeline's parser (grok/JSON) to the actual format, confirm the **index filter** and **retention**, and re-`apply`. Watch cost — index only what you'll query.

Ingestion / host count higher than expected (cost)

- **Symptom:** Datadog bill or host count climbs after a wave.
- **Cause:** Agents on decommissioned hosts still counted, all products enabled everywhere, or all logs indexed.
- **Fix:** Run `OBSDDOG-Report.ps1` to list reporting hosts and ingestion; remove agents from dead hosts, disable unneeded products per host, and tighten log indexing. Roll out further waves more selectively. *Plan/pricing is a Datadog-side decision — verify in Datadog's docs.*

If a failure isn't covered here, capture `datadog-agent status`, the `terraform plan` output, the integration status page, and the `OBSDDOG-Validate.ps1` / `OBSDDOG-Report.ps1` output before changing anything — most issues are a wrong site/key, a mis-scoped tag, or an unconnected integration.

7. Best Practices — Datadog Deployment for Hybrid & Cloud Estates

Keep the deployment unified, cheap, and secretless. These habits separate a Datadog estate people trust from a noisy, expensive one nobody tunes.

Manage everything as code

- **Monitors, dashboards, SLOs, integrations, and log pipelines belong in Terraform** — reviewed as pull

requests, `plan` before `apply`, versioned in Git. A dashboard someone hand-edits in the console will drift and get clobbered on the next apply; if a change is worth keeping, put it in code.

- **Pin the provider and the agent version** and re-verify before a big rollout — both change.
- **Store Terraform state in a secured backend**, never in the repo (it can contain resource metadata).

Never put keys in code

- The **API key** (agent) and **API+APP keys** (provider) come from a **secret store / env vars** at

runtime — never in `.tf`, images, installer scripts, or committed state. Run **secret scanning** in CI.

- Integration credentials (Azure app-registration secret, AWS role) are **referenced**, not inlined.
- Scope automation keys to least privilege and **rotate** them; a leaked key exposes the whole estate's

telemetry and can incur cost.

Tag first — tags are the whole game

- Apply consistent `env`**,** `service`**,** `team` (and cloud/region) tags at the **agent and integration**

level. Cross-cloud dashboards, monitors, and SLOs only work if the tags line up.

- Decide the tag taxonomy **once** and enforce it in the installer defaults and Terraform variables.

Monitors that page, not spam

- **Tune on the pilot before you widen.** Start monitors scoped to `env:pilot`; only expand thresholds

that proved meaningful. A monitor that fires constantly gets muted — and then the real one is missed.

- **Route with @-handles**, not personal emails — Slack/Teams channel + on-call (PagerDuty), by team.
- Use **downtimes/mutes** for maintenance windows instead of disabling monitors and forgetting them.
- Give every monitor a clear name (`<Purpose>-<Scope>-<Stage>`), a **priority**, and a **runbook link**

in the message.

Control cost deliberately (Datadog-side decisions)

• Datadog bills on **hosts, ingestion, indexed logs, and per-product** — roll out in **waves** and watch `OBSDDOG-Report.ps1` after each. Don't enable every product on every host reflexively.

• **Index logs selectively** — parse everything, but index (retain/searchable) only what you'll query; route the rest to cheaper archives. *Retention/index and plan/SKU choices are Datadog-side decisions — verify current pricing in Datadog's docs.*

• Watch for **agent sprawl** — decommissioned hosts still counted; keep the inventory clean.

Roll out safely

• **Pilot-first, waves, plan-first.** Never install the agent estate-wide or apply monitors globally in one shot — that risks a notification storm and a cost spike.

• Keep an **uninstall / mute** path ready (the installers support removal; monitors support downtime) so a bad wave is quickly reversible.

Where to go next (adjacent catalog areas)

• **Azure Monitor & Log Analytics** (this **Observability** category) — if part of the estate is Azure-only, the first-party option is lighter and cheaper for that slice; Datadog gives the unified multicloud pane.

• **Application Performance Monitoring** — extend the APM foundation by instrumenting each service's code (a per-service development task) to get distributed traces.

• **DevOps / IaC (Terraform)** solutions — fold this Datadog Terraform into your existing IaC pipeline so observability ships with the workload, plan→approve→apply.

• **Incident response / on-call** process work — connect the tuned monitors and SLOs to a defined on-call rotation and runbooks so alerts drive action.

8. Deliverables Summary — Datadog Deployment for Hybrid & Cloud Estates

What you receive

A complete, self-guided package to deploy **Datadog across a hybrid + multicloud estate as code** — the agent on VMs/containers/Kubernetes, the Azure/AWS/Kubernetes integrations, and dashboards, SLOs, monitors, and log pipelines defined in Terraform. Everything is **plan-first and pilot-first**, and **API/APP**

keys come from a secret store and never appear in code or state.

Documentation

• **Executive Summary** — the problem (fragmented per-cloud monitoring), the five-part solution, the outcome, the boundary, and the product/pricing "verify" note (Datadog internals deferred to Datadog docs).

• **Solution Overview** — how the agent, cloud integrations, the Datadog Terraform provider, tags, monitors, SLOs, and log pipelines fit together; the components built; scope and out-of-scope.

• **Requirements Guide** — the Datadog-specific prerequisites (org + site + API/APP keys, pinned provider, an agent rollout channel, a secret store, Azure/AWS integration prerequisites, a pilot + notification targets, and the toolchain), each with Verify / Implement, on top of the category globals.

• **Implementation Guide** — ordered steps: assess → pin provider + secure backend (no keys in code) →

pilot agent install (Linux/Windows/K8s) → wire integrations → plan/apply dashboard + SLO + pilot monitors + log pipeline → validate → expand in waves, with **every object named** using the `<Purpose>-<Scope>-<Stage>` convention.

• **Testing & Validation** — the four gates (agents report + tagged, integrations connected + no keys in code, a monitor fires and routes + log pipeline parses, dashboard + SLO render) as a pass/fail matrix.

• **Troubleshooting** — symptom → root-cause → fix for the common failures (agent not reporting, integration not connecting, provider auth, monitor not firing/routing, log parsing, cost/host sprawl).

• **Best Practices** — everything-as-code, never-keys-in-code, tag-first, monitor tuning, cost control, and where to go next.

Deployable artifacts — plan-first / pilot-first, no keys in code

Artifact	Purpose	Default mode
`main.tf`	The core Terraform (Datadog provider): Azure/AWS/K8s integrations, the estate dashboard, the SLO, the three pilot monitors, and the log pipeline — provider **pinned & flagged verify**; keys read from env/secret store	Declarative (plan before apply)
`install-datadog-agent.sh`	Linux agent installer: reads the API key from the **secret store**, sets site + `env`/`service`/`team` tags, optional logs/APM; supports uninstall	Idempotent install (key from vault)
`Install-DatadogAgent.ps1`	Windows agent installer: same inputs and secret-store key sourcing as the Linux installer	Idempotent install (key from vault)

Artifact	Purpose	Default mode
`OBSDDOG-Validate.ps1`	Confirms provider auth (keys via env, not code), the expected Datadog resources exist, integrations report connected, and **no keys are in the repo/state**	Read-only
`OBSDDOG-Report.ps1`	Inventory/coverage report via the Datadog API: hosts by tag, monitors + states, SLO/error budget, ingestion snapshot for the cost plan	Read-only

The installers and Terraform read the **API/APP keys from a secret store** at runtime; the PowerShell helpers are **read-only**; every Datadog-resource change is a reviewed `terraform plan` **before** `apply` — **no credentials or secrets live in the artifacts or state**.

Outcomes

- **One pane of glass** over Azure + AWS + on-prem, correlated by consistent tags.
- **Observability as versioned code** — monitors, dashboards, SLOs, and pipelines reviewed as PRs.
- **Tuned alerting** — thresholds proven on a pilot before they page the estate, routed to on-call.
- **Secretless** — keys in a vault/secret store; the repo and state are safe to share.
- A **pilot-first**, wave-based, reversible rollout (revert + apply, mute/uninstall) with a coverage report.

Assumptions & boundaries

- **Datadog console internals, plan/SKU selection, and pricing** are **deferred to Datadog's docs** — the

solution deploys and manages, it does not choose your commercial plan.

- **Provider/agent versions** are pinned and flagged **"verify"** — confirm current versions first.
- **Monitor thresholds, SLO targets, on-call routing, and log retention/indexing** are the customer's/

service-owner's decisions — the guide wires them up.

- **Deep per-service APM instrumentation** is out of scope (the APM *foundation* is enabled); **Azure-only**

estates may prefer the first-party **Azure Monitor** solution in this category.

> *Delivered as a **Complete Solution Package (PDF + Word + scripts)** — the full documentation set plus all five deployable artifacts (Terraform, the Linux + Windows agent installers, and the PowerShell helpers) in a single ZIP, yours to keep and reuse on every rollout.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

Install-DatadogAgent.ps1

Purpose: Installs the Datadog Agent on a Windows host as part of an as-code rollout. The API key is read from the ENVIRONMENT (populated from your SECRET STORE) — it is NEVER hard-coded or printed. Sets the Datadog site and base tags (env/service/team), and optionally enables logs/APM. Supports an -Uninstall path for rollback. HOW TO RUN (key comes from your secret store, not the command line history) $env:DD_API_KEY = (Read-From-Secret-Store) # e.g. Get-AzKeyVaultSecret ... -AsPlainText powershell -ExecutionPolicy Bypass -File .\Install-DatadogAgent.ps1 `-Site datadoghq.com -Env pilot -Service demo -Team platform -EnableLogs -EnableApm Uninstall: .\Install-DatadogAgent.ps1 -Uninstall EXPECTED RESULTS The agent MSI is installed and the service started; & "$env:ProgramFiles\Datadog\Datadog Agent\bin\agent.exe" status` shows API key OK and the host reports to Datadog with the given tags. NOTES White-label. NO SECRETS IN CODE: DD_API_KEY is taken from the environment/secret store and never printed. Requires an elevated (Administrator) session. Verify the current agent version / MSI URL for Windows in Datadog's docs. .EXAMPLE .\Install-DatadogAgent.ps1 -Env pilot -Service demo -Team platform -EnableLogs > [CmdletBinding()] param([string]$Site = 'datadoghq.com', # verify your org's site [string]$Env = 'pilot', [string]$Service = 'demo', [string]$Team = 'platform', [switch]$EnableLogs, [switch]$EnableApm, [switch]$Uninstall)

OBSDDOG-Report.ps1

Purpose: Produces the coverage/cost evidence for the deployment: hosts reporting (grouped by env/service tag), the monitors and their current states, the SLO status/error budget, and a host-count snapshot to watch against the cost plan. Use it to confirm coverage climbs per rollout wave and to spot agent sprawl (dead hosts still counted). Read-only — queries the Datadog API only. HOW TO RUN $env:DD_API_KEY = (Read-From-Secret-Store); $env:DD_APP_KEY = (Read-From-Secret-Store) powershell -ExecutionPolicy Bypass -File .\OBSDDOG-Report.ps1 -Site api.datadoghq.com -CsvFolder C:\Temp\obsddog EXPECTED RESULTS Console summary + optional CSVs: hosts by tag, monitor states, SLO status, and total host count. Compare across waves; investigate any unexpected host-count growth (cost). NOTES White-label. Read-only. Keys read from the environment/secret store, never printed. Ingestion/host pricing is a Datadog-side decision — verify in Datadog's docs. .EXAMPLE .\OBSDDOG-Report.ps1 -Site api.datadoghq.com > [CmdletBinding()] param([string]$Site = 'api.datadoghq.com', # verify your org's site [string]$CsvFolder)

How to run: $env:DD_API_KEY = (Read-From-Secret-Store); $env:DD_APP_KEY = (Read-From-Secret-Store) powershell -ExecutionPolicy Bypass -File .\OBSDDOG-Report.ps1 -Site api.datadoghq.com -CsvFolder C:\Temp\obsddog EXPECTED RESULTS Console summary + optional CSVs: hosts by tag, monitor states, SLO status, and total host count. Compare across waves; investigate any unexpected host-count growth (cost). NOTES White-label. Read-only. Keys read from the environment/secret store, never printed. Ingestion/host pricing is a Datadog-side decision — verify in Datadog's docs. .EXAMPLE .\OBSDDOG-Report.ps1 -Site api.datadoghq.com > [CmdletBinding()] param([string]$Site = 'api.datadoghq.com', # verify your org's site [string]$CsvFolder)

OBSDDOG-Validate.ps1

Purpose: Post-deployment gate check (05-testing). Confirms: the Datadog API/APP keys are present in the ENVIRONMENT (not in code) and authenticate; the expected Datadog resources exist (the estate dashboard, the SLO, the three pilot monitors, the log pipeline); the Azure/AWS integrations report connected; and — the security gate — that NO API/APP keys or secrets are committed in the repo. Read-only — queries the Datadog API and scans local files; makes no changes. HOW TO RUN $env:DD_API_KEY = (Read-From-Secret-Store); $env:DD_APP_KEY = (Read-From-Secret-Store) powershell -ExecutionPolicy Bypass -File .\OBSDDOG-Validate.ps1 ` -Site api.datadoghq.com -RepoPath . EXPECTED RESULTS PASS/FAIL per gate. All PASS => deployment healthy and secret-clean; proceed to Expand. Any FAIL => see 06-troubleshooting. A key found in the repo is a CRITICAL fail — rotate it immediately. NOTES White-label. Read-only. Keys are read from the environment/secret store and never printed. .EXAMPLE .\OBSDDOG-Validate.ps1 -Site api.datadoghq.com -RepoPath . > [CmdletBinding()] param([string]$Site = 'api.datadoghq.com', # verify your org's site (api.datadoghq.eu, etc.) [string]$RepoPath = '.')

How to run: $env:DD_API_KEY = (Read-From-Secret-Store); $env:DD_APP_KEY = (Read-From-Secret-Store) powershell -ExecutionPolicy Bypass -File .\OBSDDOG-Validate.ps1 ` -Site api.datadoghq.com -RepoPath . EXPECTED RESULTS PASS/FAIL per gate. All PASS => deployment healthy and secret-clean; proceed to Expand. Any FAIL => see 06-troubleshooting. A key found in the repo is a CRITICAL fail — rotate it immediately. NOTES White-label. Read-only. Keys are read from the environment/secret store and never printed. .EXAMPLE .\OBSDDOG-Validate.ps1 -Site api.datadoghq.com -RepoPath . > [CmdletBinding()] param([string]$Site = 'api.datadoghq.com', # verify your org's site (api.datadoghq.eu, etc.) [string]$RepoPath = '.')

install-datadog-agent.sh

Purpose: Installs the Datadog Agent on a Linux host as part of an as-code rollout. The API key is read from the ENVIRONMENT (populated from your SECRET STORE) — it is NEVER hard-coded or echoed. Sets the Datadog site and the base tags (env/service/team), and optionally enables logs and APM. Supports an --uninstall path for rollback. HOW TO RUN (key comes from your secret store, not the command line history) export DD_API_KEY="$(read-from-secret-store)" # e.g. az keyvault secret show ... / vault read DD_SITE=datadoghq.com \ ./install-datadog-agent.sh --env pilot --service demo --team platform --logs --apm Uninstall: ./install-datadog-agent.sh --uninstall EXPECTED RESULTS The agent is installed and started; `sudo datadog-agent status` shows API key OK and the host reports to Datadog with the given tags. Re-runnable (idempotent-ish): re-running updates tags/config. NOTES White-label. NO SECRETS IN CODE: DD_API_KEY is taken from the environment/secret store and never printed. Uses the official Datadog install script over HTTPS. Requires root/sudo. Verify the current agent major version / install method for your distro in Datadog's docs. set -euo pipefail

main.tf

Purpose: Manages the Datadog observability stack AS CODE via the Datadog provider: - Cloud integrations: Azure (datadog_integration_azure), AWS (datadog_integration_aws) - Estate dashboard: OBSDDOG-Dashboard-EstateOverview - SLO: OBSDDOG-SLO-APIAvailability - Monitors (PILOT): OBSDDOG-Monitor-HostUnreachable-Pilot / -CPUHigh-Pilot / -K8sPodCrashloop-Pilot - Log pipeline: OBSDDOG-Pipeline-AppLogs Agent rollout is handled by the installer scripts (install-datadog-agent.sh / Install-DatadogAgent.ps1) and, on Kubernetes, the official Helm chart — NOT baked in here. HOW TO RUN (plan/what-if FIRST — never apply blind) export DD_API_KEY="$(read-from-secret-store)" # NEVER hard-code — from a secret store export DD_APP_KEY="$(read-from-secret-store)" terraform init terraform plan -out tfplan # review the what-if in a pull request terraform apply tfplan # pilot scope first (env:pilot),

then expand in waves EXPECTED RESULTS plan creates the integrations, dashboard, SLO, the three PILOT monitors, and the log pipeline — and NOTHING unexpected. Thresholds/targets/handles are variables — set them to YOUR SLOs. NOTES White-label. *** NO SECRETS IN CODE** *: DD_API_KEY / DD_APP_KEY come from the environment / secret store; the Azure client secret and AWS role are passed via variables sourced from the secret store — never inline, never committed. Store Terraform STATE in a secured backend, not the repo. Provider version is PINNED and FLAGGED "verify" — confirm the current supported DataDog/datadog provider version against the registry/Datadog docs before you apply.